Exercício em duplas (10 min): qual método é chamado? (1/3)

Para cada trecho: **(a)** qual é a saída? **(b)** a decisão foi tomada na **compilação** ou na **execução**?

1 · Java

```
class Animal {
  String som() { return "..."; }
}
class Cachorro extends Animal {
  String som() { return "au"; }
}
Animal x = new Cachorro();
System.out.println(x.som());
```

2 · C++

```
struct A {
  void f() { cout << "A"; }
};
struct B : A {
  void f() { cout << "B"; }
};
B b; A *p = &b;
p->f();
```



Java: a saída é `au`. Como `x` aponta para um objeto `Cachorro`, é o método dessa classe que roda, mesmo com a variável declarada como `Animal`. Vinculação dinâmica: quem escolhe o método é o objeto real, em tempo de execução.



C++: a saída é `A`. O método `f` não foi declarado como `virtual`, então o compilador olha só para o tipo do ponteiro (`A*`) e fixa a chamada ali. Vinculação estática.

Exercício: qual método é chamado? (2/3)

Para cada trecho: **(a)** qual é a saída? **(b)** a decisão foi tomada na **compilação** ou na **execução**?

3 - Java

```
class A { String nome = "A";
  String getNome() { return nome; } }
class B extends A { String nome = "B";
  String getNome() { return nome; } }
A x = new B();
System.out.println(x.nome + " " + x.getNome());
```

4 - Python

```
class Contador:
    total = 0
    def __init__(self):
        Contador.total += 1
        self.id = Contador.total

a = Contador(); b = Contador()
print(a.id, b.id, a.total)
```



Java: a saída é `A B`. Campos não têm polimorfismo: `x.nome` usa o tipo declarado (`A`), e o campo `nome` de `B` apenas esconde o de `A`, sem substituí-lo. Já `getNome()` é um método e é resolvido em tempo de execução, devolvendo o valor do `B`. Esse é um bom motivo para deixar os campos privados.



Python: a saída é `1 2 2`. `total` é uma variável de classe, com uma única cópia compartilhada por todos os objetos, enquanto `id` é de instância, um para cada objeto. Por isso `a.total` acaba encontrando o `total` da classe, que já chegou a 2.

Exercício: qual método é chamado? (3/3)

Para cada trecho: **(a)** qual é a saída? **(b)** a decisão foi tomada na **compilação** ou na **execução**?

5 · Java

```java
class A {
  static String quem() { return "A"; }
}
class B extends A {
  static String quem() { return "B"; }
}
A x = new B();
System.out.println(x.quem());
```

6 · Go

```go
type Animal struct{}
func (Animal) Som() string { return "..." }
func (a Animal) Falar() string {
    return "faz " + a.Som()
}
type Cao struct{ Animal }
func (Cao) Som() string { return "au" }

fmt.Println(Cao{}.Falar(), Cao{}.Som())
```



Java: a saída é `A`. Métodos `static` não são sobrescritos, e sim escondidos. A chamada é resolvida na compilação pelo tipo declarado (`A`). O próprio compilador ou a IDE costuma avisar que o correto seria chamar `A.quem()`.



Go: a saída é `faz ... au`. Embedding não é herança. Dentro de `Animal.Falar`, a chamada `a.Som()` usa o `Som` de `Animal`, porque não existe despacho dinâmico entre a struct embutida e a que a contém. Já `Cao{}.Som()` chama o `Som` de `Cao` diretamente. Para ter polimorfismo em Go, é preciso usar uma interface.